

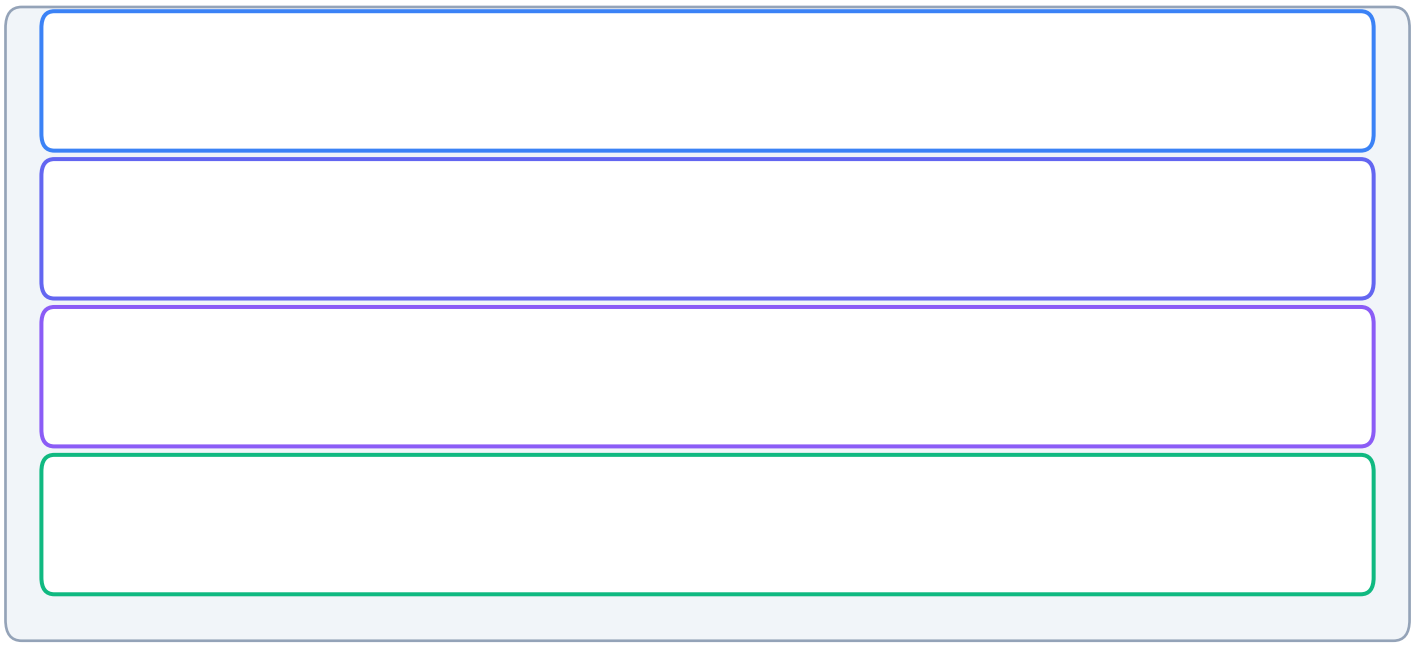
erver-Patching-Tool

1. Executive Summary & Problem Framing

Modern cloud backends face increasing operational complexity: high-frequency deployments, microservice inter-dependencies, and sudden transient or logical failures create cascading outages. Traditional automated remediation approaches are brittle, hard-coded rule scripts, while naive LLM-based autonomous coding creates immense risk: hallucinated fixes, unvetted secret exposures, regression bugs, and unauthorized production commits.

Project UMBRELLA addresses this gap by architecting a Risk-Aware Multi-Agent Framework where a Hybrid Agentic RAG system provides evidence-grounded diagnosis, while specialized agents coordinate under a strict deterministic policy gate. To establish a rigorous architectural foundation, we acquired and curated four official open-source codebases from Infosys (published under Apache-2.0), extracting proven orchestration patterns, guardrails, and self-healing loops.

Core Logical Architecture: UMBRELLA Self-Healing Pipeline



The Provenance & Boundary Invariant

- Third-Party Reference Layer: The curated Infosys materials serve as architectural and workflow design references. They do NOT represent internal proprietary production telemetry or customer-specific incident records.
- Deterministic Security Boundary: Application secrets, credentials, and tokens are NEVER exposed to the LLM or stored in RAG vector embeddings. Configuration repairs occur via sanitized references and human-in-the-loop gates.
- Verification Precedence: No model-generated code change can self-authorize. All commits must pass programmatic test suites in an isolated sandbox, policy verification, and human operator review before merging.

2. Infosys Agentic Foundry (IAF) — Orchestration & Safeguards

Infosys Agentic Foundry (V1.9.0, May 2026) is Infosys Topaz's flagship open-source agent framework. It provides production-tested paradigms for controller loops, multi-agent coordination, human-in-the-loop gates, and sandboxing.

A. Controller Flow & Execution Loop

The IAF controller implements a rigorous 5-stage sequential processing cycle:

1. Orchestration: Request routing, priority evaluation, and agent assignment.
2. Context & Memory: Assembly of short-term conversational buffer and long-term knowledge.
3. Reasoning: Execution of prompt-engineered reasoning loops (ReAct or ReAct-Critic).
4. Tool Execution: Deterministic tool call dispatching with pre-execution authorization.
5. Verification & Response: Synthesizing grounded outputs and updating audit trails.

UMBRELLA Adoption: Used directly to structure the coordination between Root Cause Intelligence, Remediation Strategy, and Safety Guardian.

B. Multi-Agent Templates & Coordination Patterns

IAF provides standardized agent architectural archetypes:

- ReAct & ReAct-Critic: Interleaves reasoning and acting with an adversarial critic to check hallucination.
- Planner-Executor-Critic: Decomposes complex tasks into verified multi-step subtasks.
- Meta-Orchestrator: Coordinates specialist worker agents across functional boundaries.

UMBRELLA Adoption: Specialist agents operate on dedicated roles with strict schema contracts, preventing uncontrolled autonomous sprawl.

C. Sandboxed Execution & Human-in-the-Loop (HITL) Gates

IAF enforces sandboxed tool execution using Docker containers or nsjail namespaces with strict compute, memory, and network quotas. Furthermore, high-impact tool invocations trigger mandatory HITL review.
UMBRELLA Adoption: Execution Agent creates patches in isolated Git worktrees and executes pytest suites within locked network containers. Rollbacks are automated upon any test regression.

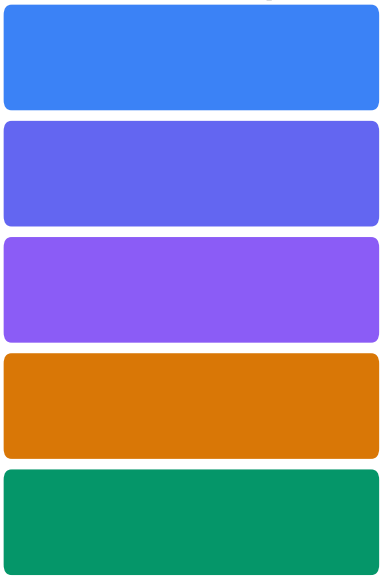
3. Infosys Responsible AI Toolkit — Guardrails & Grounding

The Infosys Responsible AI Toolkit provides enterprise-grade safety modules covering hallucination scoring, PII detection/redaction, prompt safety moderation, and model explainability APIs. In Project UMBRELLA, these capabilities form the pre-ingestion and pre-action safety boundary.

4. Intelligent Bot Management & Server Patching Tool

Self-healing in production backends requires robust state machine management, execution tracking, and failure classification. We synthesized operational workflows from Infosys Intelligent Bot Management and Server-Patching-Tool.

The Closed-Loop Self-Healing Lifecycle



Continuously ingests HTTP 5xx spikes, database connection timeouts, and deadlocks via probes.

Classifies failure into Root Cause taxonomy: Config Error, Schema Mismatch, Concurrency, Dependency.

Remediation Strategy generates candidate diff with rollback script and test coverage requirement.

Safety Guardian evaluates risk level. If Low -> Isolated Worktree Test; If High -> Request HITL Approval.

Runs regression test suite and active health check. On failure: immediate git rollback and alert.

Server-Patching-Tool: API Patterns for Execution Tracking

The Server-Patching-Tool codebase provides an asynchronous job execution model: endpoints initiate patch workflows, track state in relational databases, and expose status endpoints (`/job/{id}/status`, `/execution/verify`).

UMBRELLA Implementation:
• Remediation Execution Agent exposes structured job states: `PROPOSED`, `SANDBOX_TESTING`, `AWAITING_APPROVAL`, `APPLIED`, `VERIFIED_HEALTHY`, and `ROLLED_BACK`.
• State persistence guarantees that interrupted repairs do not leave the backend in a corrupted or unverified state.

5. Hybrid Agentic RAG Blueprint for Project UMBRELLA

Phase 2 will construct UMBRELLA's Hybrid Agentic RAG engine. The design combines lexical precision (for exact error messages, API routes, and config keys) with dense vector semantic search and outcome memory.

6. Provenance Manifest & Credential Security Audit

All ingested codebases were retrieved from official Infosys GitHub repositories, verified under Apache License 2.0, and subjected to automated credential and secret screening prior to curation staging.

Repository Provenance & Curation Matrix

Security Verification & Next Execution Milestones

1. Phase 1 Accomplished: All 4 repositories acquired in isolated staging; 0 leaked credentials found; high-value architecture, guardrail, and remediation code structured into `umbrella-curated-reference/` with SHA256 manifest.
2. Phase 2 (Immediate Next): Initialize the Hybrid Agentic RAG engine with BM25 keyword index, FAISS vector index, and metadata filters over the curated reference corpus.
3. Phase 3: Implement the 5 Specialist Agents (Root Cause, Visualization, Remediation, Execution, Safety Guardian) connecting them to the RAG knowledge service via typed Pydantic contracts.
4. Phase 4: Construct the Agent 6 Outcome-Aware Self-Learning engine evaluated against reproducible GrowSphere backend fault scenarios.